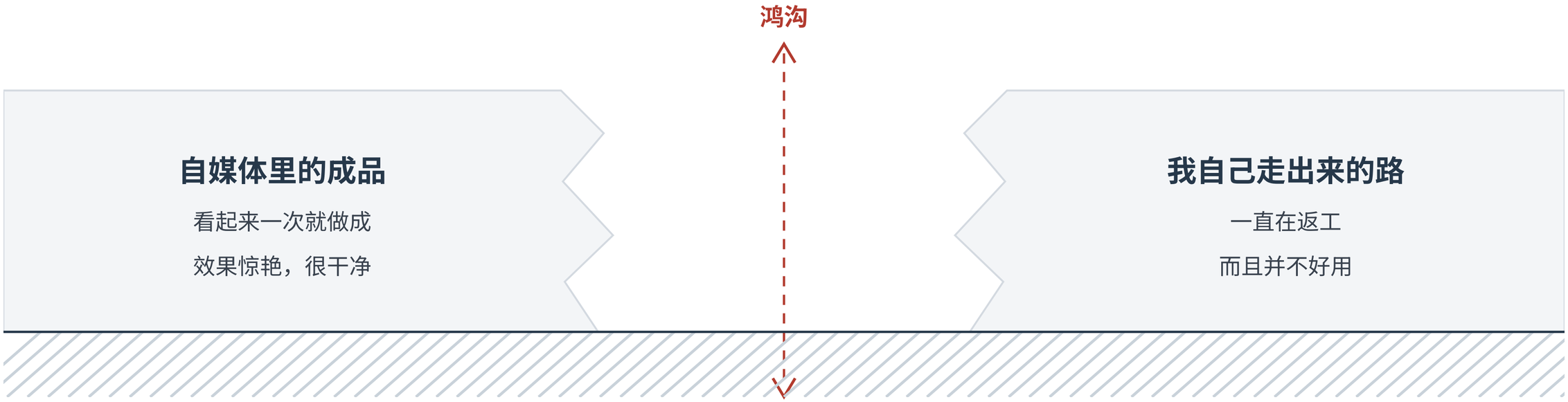
自媒体与自我应用的鸿沟



同一套方法，两个结果

自媒体给你看到的

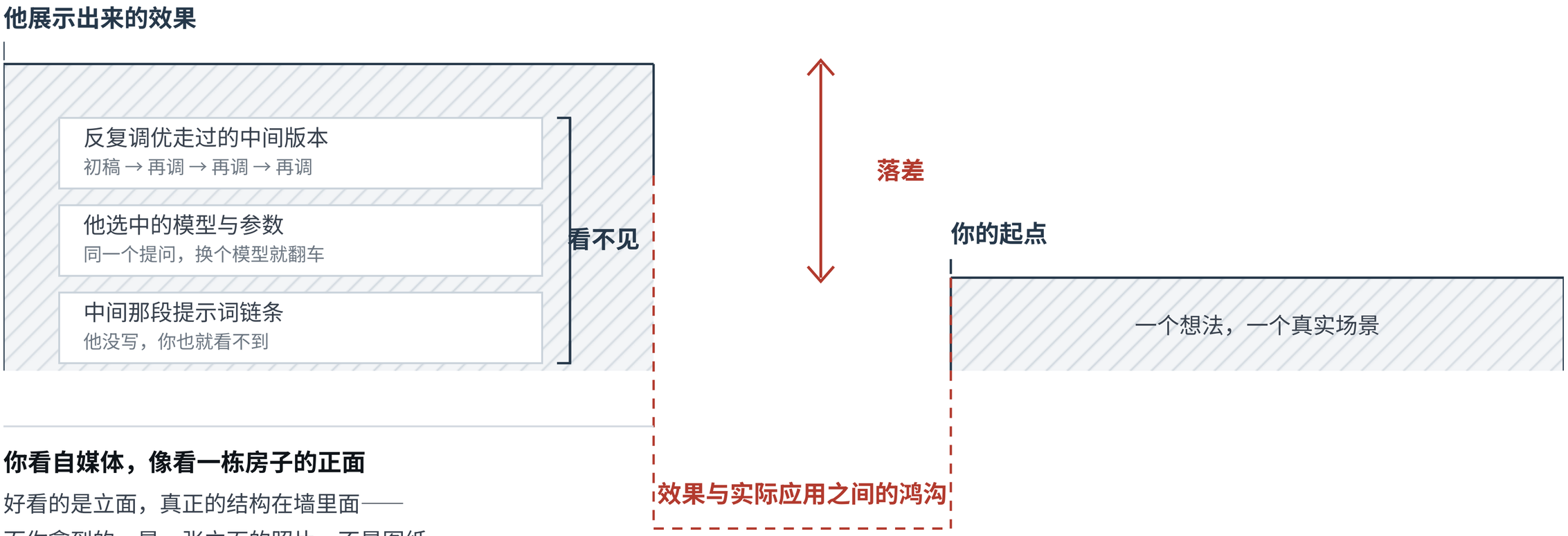
- 效果惊艳，像是随手就能做
- 流程清楚，一步跟着走
- 换个场景应该也没问题
- 一次就成，方法即答案



你实际走出来的

- 做出来的东西根本不能直接用
- 而且非常丑
- 过程全靠猜，不知道他在优化什么
- 你的场景，完全跑不通

把画面剖开看：鸿沟在哪里



你看自媒体，像看一栋房子的正面

好看的是立面，真正的结构在墙里面——

而你拿到的，是一张立面的照片，不是图纸

原因一：自媒体上展现的，是优化后的结果

作者实际走的路

第一版
不对，重来

→

调了一轮
还是不对

→

又调了一轮
差一点点

→

继续改
提示词换了三版

→

还有一堆没发的
中间版本全部丢掉

整了三天
多次调优、多次迭代之后，才有了那一条

这三天里真正发生的事

提示词改了三版，素材重做过

换过模型，结构推翻重来

每一版都推倒，删掉的比留下的多

这些，一条视频里都看不见

→

自媒体上，你看到的成品

一个已经调好的结果

可以直接抄、直接照着用

过程没有出现，代价没有出现

你抄到的，只是这条线的最后一格

所以你看到的不是方法，是方法的 **成品**

原因二：方法里还装着一个黑箱

你抄到的

一个最终效果

和一句操作说明

黑箱

你只知道它能做出这个效果

不知道里面装了什么

用的什么模型

换掉就可能翻车

中间的提示词

根本没告诉你

素材与上下文

也不在你手上

作者自己知道这些，只是没说

你只能看到外面那一层

你面对的

一个做不出

同样效果的黑箱

黑箱的代价

看不见的部分只能靠猜，猜错就重做

破箱的方式

把它拆开：模型、提示词、素材，一个一个试过去

两个原因合在一起，就成了那道鸿沟

原因一

展现的是优化后的结果

你看到的是最后一格

代价和过程都不在里面

原因二

方法里装着一个黑箱

模型、提示词、素材都看不见

你只能对着结果猜过程

鸿沟

效果与实际应用之间

你照着做

过程无法还原

结果面目全非

于是得出结论：

“这方法不行”

真正该问的

成品背后

有多少我看不见的东西

以及我能不能

把它们一步步补齐

把成品换成方法：什么能带走

能带走

它的思路
为什么这么拆、为什么要这一步

它的流程骨架
先做什么后做什么，顺序为什么这样排

它的选型理由
这一步为什么用这个工具而不是别的

它踩过的坑
哪些路他试过，你就可以直接跳过

带不走

那份成品本身
它长在他的场景里，换个场景就不成立

那串提示词原文
你不知道它在中间改过多少遍

他没说出口的前提
素材、数据、账号，都是他的不是你的

三天的迭代时间
这部分永远得由你自己付出来

拿走思路和流程，把迭代的时间还给自己

大多数人的正确用法，是一个循环

一、先做一个东西

哪怕很粗糙，先跑通一个
属于你自己的场景

这一条你已经在做了

二、在做的过程中迭代优化

边做边调，不等完美方案
出来再往下走一步

这一条被大多数人跳过了

三、不断与智能体协同

把它当同事，不是当答案机
对话、追问、一起改

这一条是拉开差距的地方

回到场景，继续迭代

等一个方法

一次适配自己
从此不用再管

期待本身就是鸿沟的来源

不是先找到方法再去应用，而是先做出来，再在做的过程中把它调成你自己的

看完之后，三个动作就能开始

一

挑一个自媒体案例

先问清楚三件事：

他用的什么模型

提示词大致是什么结构

中间迭代了几轮

当成拆解题，不要当抄写题

二

在自己的场景跑一遍

拿你自己的素材和目标

先跑通一个粗糙版本

丑一点没关系，能跑就行

这一步不跳过

跑通，才有得迭代

三

把协同当成主线

每做一步都回头问：

哪里不对，为什么

能不能换个模型或改写

再来一轮

迭代出来的东西才是你的

这三个动作，就是把成品拆开、把自己放进去、然后反复打磨的过程

它不保证你一次就做出那种效果，但它保证你做出来的东西是自己的、能用

自媒体给你的是别人的终点，
你要走的是自己的全过程

看到的一条

别人的成品
一个结果，看不到过程

要走的一条

你自己的全过程
做出来、调出来、协同出来

- 别拿别人的成品，当成自己的起点
- 看不见的部分，就自己去拆、去试、去补
- 真正拉开差距的，是在做的过程中反复迭代

鸿沟不会因为你换了个更聪明的方法就消失，它只会在你真正动手的过程中被填上